Name : Vaishnavi Eknath Avhad
Class : D15C
Roll No. : 14

Practical 10

Aim : Explore and implement Autoencoders for image denoising.

1. Dataset Source :

The dataset used for this experiment is the MNIST Database of Handwritten Digits.

- Source Link : MNIST Dataset - Keras Documentation

---

2. Dataset Description

The MNIST dataset is used here in an unsupervised manner. We ignore the labels and focus on the image pixels to train the model to reconstruct clean images from noisy versions.

Property	Details
Input Data	28×28 grayscale images of digits 0-9.
Training Samples	60,000 images.
Preprocessing	Normalization (0 to 1) and injection of Gaussian noise.
Output Data	Reconstructed "clean" version of the noisy input.

---

3. Mathematical Formulation

A. The Autoencoder Architecture

An autoencoder consists of two main parts : the Encoder and the Decoder.

1. Encoder (h = f(x)): Maps the input x to a lower-dimensional latent representation h.
2. Decoder (r = g(h)): Attempts to reconstruct the input from the latent space.

B. Denoising Objective

In a Denoising Autoencoder (DAE), the input is a corrupted version of the data (tilde{x} = x + noise). The network is trained to minimize the reconstruction loss between the output and the *original* clean image x :

$$L(x, \hat{x}) = ||x - g(f(\tilde{x}))||^2$$

By forcing the model to recover the original image, it learns to identify and remove noise patterns while preserving structural features.

---

4. Algorithm Limitations

1. Blurry Reconstructions : Autoencoders using Mean Squared Error (MSE) loss often produce slightly blurry images because the model averages out pixel values to minimize error.
2. Sensitivity to Noise Type : A model trained on Gaussian noise might not perform well on "Salt and Pepper" noise or structured artifacts.
3. Capacity Bottleneck : If the latent space (bottleneck) is too large, the model might just "memorize" the noise instead of learning features.
4. Computational Cost : Convolutional Autoencoders (CAE) require significant GPU resources due to multiple convolution and upsampling layers.

---

5. Methodology

Code Implementation

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.models import Model
# ----------------- 1. PREPARE DATA -----------------
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
# Add Gaussian Noise
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

```
# ----------------- 2. BUILD DENOISING AUTOENCODER -----------------
input_img = Input(shape=(28, 28, 1))
# Encoder: Compressing the image
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x)
# Decoder: Reconstructing the image
x = Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
# ----------------- 3. TRAINING -----------------
print("Training Autoencoder...")
history = autoencoder.fit(x_train_noisy, x_train,
            epochs=10,
            batch_size=128,
            shuffle=True,
            validation_data=(x_test_noisy, x_test),
            verbose=1)
# ----------------- 4. VISUALIZATION -----------------
decoded_imgs = autoencoder.predict(x_test_noisy)
n = 10
plt.figure(figsize=(20, 6))
for i in range(n):
    # Original
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
```

```
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    # Noisy
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
    plt.title("Noisy")
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    # Denoised
    ax = plt.subplot(3, n, i + 1 + 2*n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Denoised")
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()
```

---

6. Performance Analysis

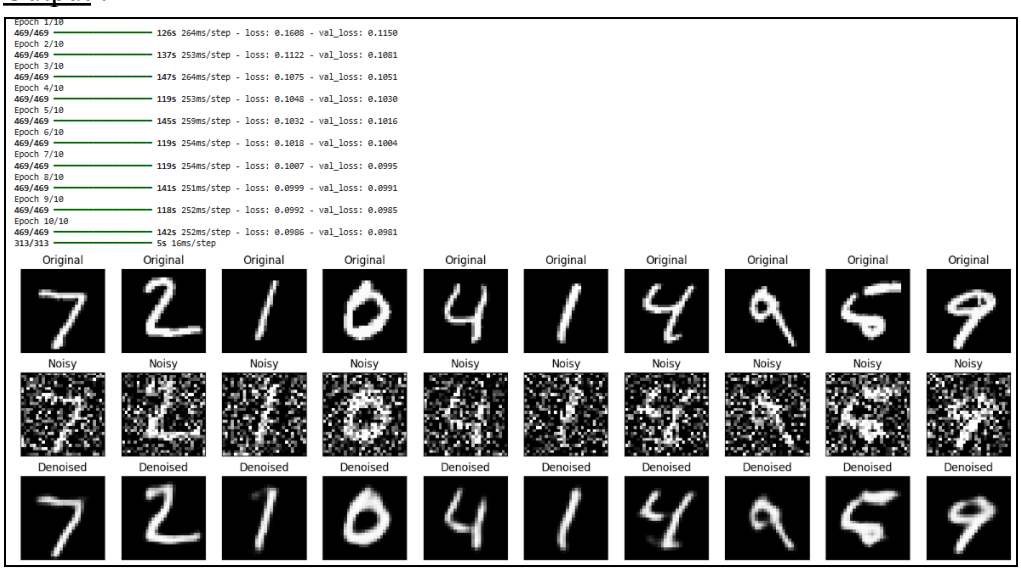
A. Loss Convergence

During training, the binary_crossentropy loss decreases as the model learns to ignore the random Gaussian pixels. A steady gap between training and validation loss suggests the model has learned general features of digits rather than memorizing the specific noise in the training set.

B. Visual Comparison

- Noisy Image : The digits are barely recognizable due to the high noise factor (0.5).
- Denoised Image : The model effectively "smooths" out the noise. While some fine edges are slightly blurred, the digit's structure is clearly restored, proving the effectiveness of the convolutional bottleneck.

Output :



7. Hyperparameter Tuning

Trial	Noise Factor	Filters in Conv2D	Result	Observation
Trial 1	0.2	16	High Accuracy	Clean reconstruction; low noise is easy to filter.
Trial 2	0.5	32	Good Balance	Optimal: Handles heavy noise without losing digit shape.
Trial 3	0.8	64	Lower Quality	Reconstruction becomes "ghostly" or loses parts of the digit.

Tuning Interpretation : Increasing the number of filters (Trial 2) allows the model to learn more complex spatial features, which is necessary when the noise level is high. However, if the noise factor is too extreme (Trial 3), even a deeper network struggles to recover the signal.

---

8. Conclusion

In this practical, a Convolutional Autoencoder was successfully implemented for image denoising. The experiment demonstrated that by utilizing a bottleneck architecture, the model can be forced to learn the most salient features of a dataset. We concluded that the denoising capability is a powerful form of unsupervised feature learning, where the model learns the "concept" of a digit to effectively separate it from random noise.